Universidad Simón Bolívar
Departamento de Computación y Tecnología de la Información.
CI4835 - Redes de Computadoras I
Integrantes:
Ricardo Garcia 20-10274
Enyerber Silva 18-11100

PROTOCOLO DE CAPA DE APLICACIÓN ORIENTADO A TEXTO CON SU CLIENTE

1 Introducción

En este proyecto se nos pidió implementar un protocolo de capa de aplicación orientado a texto con su respectivo cliente, que trabaja con la API de sockets TCP, usamos la biblioteca `include <sys/socket.h>`, que es de POSIX. Ojo, se declara y jura que se usaron agentes de IA para debuggear y permitir la conexion y creacion de sockets, fue especialmente util poder hacer el poll y manejo de direcciones de memoria, usamos agentes como Claude Opus 5.5 y Fable 5.1, asi como Gemini.

2 Modelo de Concurrencia usado

El sistema usa una única llamada al sistema dentro de un bucle principal, llamando a la función `poll()`. Esto quiere decir que estamos usando un único hilo, lo que nos evita tener que lidiar con problemas de la sección critica, o Mutexes y Signals. Usar `poll()` hace que la implementación sea algo mas sencilla, ademas, `poll()` usa un vector dinámico, que permite escalar la cantidad de clientes en RAM a medida que vaya siendo necesario.
De manera general, `poll()` funciona de la siguiente manera: permite que un proceso pueda monitorear un socket TCP (bueno, realmente permite monitorear un file descriptor) de manera simultanea, buscando quien esta listo para una operación de I/O, y pone a dormir el hilo del servidor mientras no haya nadie pidiendole nada (ojo sin bloquearlo). Una consecuencia directa de este tipo de implementación es que, si un cliente realiza una operación demasiado pesada, los otros tendrían que esperar a que termine para poder realizar su pedido.

3 Problema de la delimitación de mensajes

EL problema es el siguiente: un cliente no puede escribir un mensaje absurdamente largo, debe tener un limite, para poder resolver esto creamos una constante llamada `MAX_LINEA`, que permite un máximo de linea de 512, que es lo mismo que el tamaño del buffer que usamos. Ademas, cada linea de texto que manda el servidor termina con `/r/f`, de manera que el servidor sepa interpretar el final de una linea. Ademas, si la respuesta es demasiado larga, el servidor la acorta, permitiendo lo máximo que su buffer tolere.

4 Delimitaciones conocidas

Para empezar, se tuvieron que crear numerosas funciones auxiliares que permitieran validar las salidas y entradas de datos, clientes. También se tuvo que construir una función que permita manejar las desconexiones abruptas de los clientes del servidor. Se implemento la siguiente función, capaz de reconocer y manejar las salidas abruptas de los clientes del servidor:



```
void desconectar_cliente(int fd) {
    if (clientes.count(fd) == 0) return;

    Cliente c = clientes[fd];
    if (!c.es_http && !c.apodo.empty()) {
        guardar_buzon(c);
        std::string aviso = "302 " + c.apodo + " se ha desconectado\r\n";

        // Notificar y remover del mapa de canales
        for (auto it_canal = canales.begin(); it_canal != canales.end(); ) {
            auto& miembros = it_canal->second;
            auto it_m = std::find(miembros.begin(), miembros.end(), fd);
            if (it_m != miembros.end()) {
                miembros.erase(it_m);
                for (int m_fd : miembros) {
                    send(m_fd, aviso.c_str(), aviso.length(), 0);
                }
            }
            if (miembros.empty()) {
                it_canal = canales.erase(it_canal);
            } else {
                ++it_canal;
            }
        }
    }

    close(fd);
    clientes.erase(fd);

    for (auto it = fds.begin(); it != fds.end(); ++it) {
        if (it->fd == fd) {
            fds.erase(it);
            break;
        }
    }
}
```

Figure 1: Función que maneja la desconexion abrupta de un cliente

Ademas, y esto es importante, cuando usamos el comando, suponiendo que la IP a la que me conecto es localhost en el puerto 5000 :

```
nc -C localhost 5000
```

Debido a como funciona nc, luego de que el cliente se desconecte, la terminal del cliente no ser cerrara como tal, claramente indicara el mensaje de `299 Adiós :)`, y esto esta bien, ya que es un comportamiento de `nc`, ya que con este se escucha al puerto y al teclado (osea mi entrada estándar). Si este no es el comportamiento que se quiere, simplemente debemos agregar el flag `-q 0` al comando `nc`, quedando de la siguiente manera:

```
nc -q 0 -C localhost 5000
```

Ademas, en el caso de fedora, para permitir la conexion con otras maquinas, debemos abrir temporalmente un puerto en Fedora, ejecutando el siguiente comando:

```
sudo firewall-cmd --remove-port=5000/tcp --zone=public
sudo firewall-cmd --reload
```

Este comando debe ejecutarse con suma responsabilidad, ya que estas abriendo el puerto de tu PC para permitir trafico real, el comando sudo firewall-cmd –reload deshace esto. Ejecutarlo una vez se termine de probar el código.